

TABLE DES MATIÈRES

PARTIE I	– FONDEMENTS DES BASES DE DONNÉES
PARTIE II	– LE MODÈLE RELATIONNEL ET SQL
PARTIE III	– LANGAGE SQL : MAÎTRISE COMPLÈTE
PARTIE IV	– CONCEPTION ET MODÉLISATION
PARTIE V	– OPTIMISATION ET PERFORMANCE SQL
PARTIE VI	– SGBD RELATIONNELS MAJEURS
PARTIE VII	– INTRODUCTION AU NOSQL
PARTIE VIII	– NOSQL CLÉ-VALEUR
PARTIE IX	– NOSQL DOCUMENT
PARTIE X	– NOSQL COLONNE LARGE
PARTIE XI	– NOSQL GRAPHE
PARTIE XII	– SGBD MULTI-MODÈLES ET NEWSQL
PARTIE XIII	– TRANSACTIONS, ACID ET CAP
PARTIE XIV	– SQL vs NoSQL : CHOISIR LE BON OUTIL
PARTIE XV	– BASE DE DONNÉES EN PRODUCTION
PARTIE XVI	– CAS PRATIQUES ET ARCHITECTURES RÉELLES

=====

PARTIE I – FONDEMENTS DES BASES DE DONNÉES

=====

1.1 QU'EST-CE QU'UNE BASE DE DONNÉES ?

Une base de données est un ensemble organisé d'informations structurées, stockées et accessibles électroniquement. Elle constitue le socle de toute application informatique moderne.

DÉFINITION OPÉRATIONNELLE :

"Une base de données est un système permettant de stocker, organiser, retrouver et manipuler des données de façon fiable, cohérente et efficace, en répondant aux besoins simultanés de multiples utilisateurs."

Sans base de données bien conçue, toute architecture logicielle s'effondre. C'est la fondation invisible de tout système d'information.

1.2 LE SGBD : SYSTÈME DE GESTION DE BASE DE DONNÉES

Un SGBD (ou DBMS en anglais : Database Management System) est le logiciel qui gère la base de données. Il fait l'intermédiaire entre les données brutes et les applications ou utilisateurs.

FONCTIONS ESSENTIELLES D'UN SGBD :

- [A] STOCKAGE
 - Écriture physique sur disque ou mémoire
 - Gestion des fichiers, pages et blocs
 - Compression et chiffrement des données
- [B] INTERROGATION
 - Interface de requêtes (SQL ou autre)
 - Moteur d'exécution et d'optimisation
 - Retour des résultats au client
- [C] SÉCURITÉ
 - Authentification des utilisateurs
 - Gestion des droits et permissions
 - Audit et traçabilité des accès

- [D] INTÉGRITÉ
 - Respect des contraintes définies
 - Validation des données à l'entrée
 - Maintien de la cohérence interne
- [E] CONCURRENCE
 - Gestion simultanée de multiples accès
 - Mécanismes de verrouillage (locks)
 - Isolation des transactions
- [F] REPRISE SUR PANNE
 - Journaux de transactions (WAL / logs)
 - Sauvegardes et restaurations
 - Haute disponibilité et réplication

1.3 HISTORIQUE ET ÉVOLUTION

ANNÉES 1960 – LES FICHIERS PLATS
 Données stockées dans des fichiers texte séquentiels.
 Aucune structure, aucune relation, duplication massive.

ANNÉES 1970 – LE MODÈLE HIÉRARCHIQUE ET RÉSEAU
 IBM introduit IMS (Information Management System).
 Les données sont organisées en arbre ou en graphe.
 Navigation rigide, inflexible, couplée au code.

1970 – LA RÉVOLUTION CODD
 Edgar F. Codd publie "A Relational Model of Data for Large Shared Data Banks".
 Naissance du modèle relationnel. Rupture intellectuelle majeure.
 Les données sont organisées en tables, reliées par des clés.

ANNÉES 1980 – L'ESSOR DU SQL
 SQL devient le langage standard (norme ISO 1987).
 Oracle, IBM DB2, Sybase, puis Microsoft SQL Server émergent.
 Le modèle relationnel domine le marché pendant 30 ans.

ANNÉES 2000 – L'ÈRE DU WEB ET DU BIG DATA
 Google, Amazon, Facebook génèrent des volumes sans précédent.
 Le modèle relationnel atteint ses limites de scalabilité.
 Naissance des bases NoSQL (Not Only SQL).

ANNÉES 2010 – LA DIVERSIFICATION
 Explosion des types de SGBD : document, graphe, colonne, clé-valeur.
 Montée du cloud : RDS, DynamoDB, CosmosDB, BigQuery.
 Apparition des bases NewSQL (scalabilité + ACID).

AUJOURD'HUI – L'HYBRIDATION
 Les frontières s'effacent. Les SGBD multi-modèles se généralisent.
 SQL revient dans les bases NoSQL (CQL, N1QL, etc.).
 Le choix n'est plus SQL vs NoSQL mais quel outil pour quel besoin.

1.4 TAXONOMIE DES SGBD

SGBD			
RELATIONNELS (SQL)	NoSQL		
	Clé-Valeur	Document	Colonne Large
	PostgreSQL MySQL Oracle SQL Server MariaDB SQLite	MongoDB Couchbase Firestore RavenDB	Cassandra HBase BigTable
	GRAPHE Neo4j / Amazon Neptune / ArangoDB		

+ NewSQL : CockroachDB, TiDB, Google Spanner, YugabyteDB

- + Séries temporelles : InfluxDB, TimescaleDB, Prometheus
- + Recherche full-text : Elasticsearch, OpenSearch, Solr
- + Multi-modèles : ArangoDB, CosmosDB, FaunaDB

PARTIE II – LE MODÈLE RELATIONNEL ET SQL

2.1 LES CONCEPTS FONDAMENTAUX DU MODÈLE RELATIONNEL

Le modèle relationnel repose sur des concepts mathématiques issus de la théorie des ensembles et de la logique du premier ordre.

TABLE (RELATION)

Unité de base du modèle. Une table représente une entité du monde réel. Chaque table a un nom unique dans la base.

Exemple :

TABLE : clients			
id	nom	email	ville
1	Diallo Alpha	a@m.com	Conakry
2	Bah Mariama	b@m.com	Kindia
3	Camara Ibou	c@m.com	Labé

LIGNE (TUPLE / ENREGISTREMENT)

Chaque ligne représente une instance unique de l'entité. Appelée aussi "enregistrement" ou "row".

COLONNE (ATTRIBUT)

Chaque colonne représente une propriété de l'entité. Elle a un nom, un type de données, et des contraintes éventuelles.

DOMAINE

L'ensemble des valeurs possibles pour un attribut.
Exemple : le domaine de "age" peut être les entiers entre 0 et 150.

2.2 CLÉS ET CONTRAINTES D'INTÉGRITÉ

CLÉ PRIMAIRE (PRIMARY KEY)

Identifiant unique de chaque ligne. Ne peut être NULL.
Une table n'a qu'une seule clé primaire (simple ou composite).

Exemple : id INTEGER PRIMARY KEY

CLÉ ÉTRANGÈRE (FOREIGN KEY)

Référence la clé primaire d'une autre table.
Garantit l'intégrité référentielle : pas de référence vers un enregistrement inexistant.

Exemple : client_id INTEGER REFERENCES clients(id)

CLÉ CANDIDATE

Tout attribut (ou groupe) capable d'identifier uniquement une ligne.
La clé primaire est choisie parmi les clés candidates.

CLÉ COMPOSITE

Clé primaire formée de plusieurs colonnes combinées.
Utilisée dans les tables d'association (many-to-many).

CONTRAINTES COURANTES

- NOT NULL : la valeur ne peut pas être vide
- UNIQUE : pas de doublon dans la colonne
- CHECK : condition booléenne sur la valeur
- DEFAULT : valeur par défaut si non renseignée

2.3 LES RELATIONS ENTRE TABLES

UN À UN (1:1)

Un enregistrement d'une table correspond à exactement un enregistrement dans une autre table.

Exemple : un utilisateur a un seul profil détaillé.

UN À PLUSIEURS (1:N)

Un enregistrement d'une table correspond à plusieurs enregistrements dans une autre table.

Exemple : un client passe plusieurs commandes.

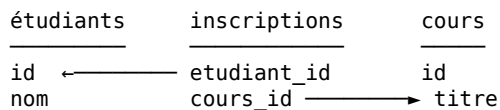
Implémentation : la clé étrangère est placée du côté "plusieurs".

PLUSIEURS À PLUSIEURS (N:M)

Plusieurs enregistrements d'une table correspondent à plusieurs enregistrements d'une autre table.

Exemple : un étudiant suit plusieurs cours, un cours a plusieurs étudiants.

Implémentation : table de jonction (table associative).



2.4 LES FORMES NORMALES (NORMALISATION)

La normalisation est le processus d'organisation des données pour réduire la redondance et améliorer l'intégrité. Elle se fait par étapes successives.

PREMIÈRE FORME NORMALE (1NF)

- Chaque cellule contient une valeur atomique (non divisible)
- Pas de groupes répétitifs dans une colonne
- Chaque ligne est unique (clé primaire définie)

VIOLATION : stocker "Conakry, Guinée" dans une colonne "adresse"

CORRECTION : séparer en colonnes "ville" et "pays"

DEUXIÈME FORME NORMALE (2NF)

- Respecte la 1NF
- Tout attribut non-clé dépend de la totalité de la clé primaire
- (S'applique uniquement aux tables à clé composite)

VIOLATION : dans (commande_id, produit_id) → nom_client dépend seulement de commande_id, pas du tout de produit_id

CORRECTION : extraire nom_client dans une table commandes séparée

TROISIÈME FORME NORMALE (3NF)

- Respecte la 2NF
- Aucun attribut non-clé ne dépend d'un autre attribut non-clé (pas de dépendances transitives)

VIOLATION : ville → code_postal → région (transitif)

CORRECTION : extraire région dans une table liée au code_postal

FORME NORMALE DE BOYCE-CODD (BCNF)

Version renforcée de la 3NF. Toute dépendance fonctionnelle $X \rightarrow Y$

implique que X est une super-clé.

PIÈGE DU PRATICIEN :

La normalisation complète n'est pas toujours l'objectif.
En analytique (datawarehouse), on dénormalise intentionnellement
pour accélérer les requêtes (modèles en étoile et flocon).

PARTIE III – LANGAGE SQL : MAÎTRISE COMPLÈTE

3.1 STRUCTURE DU SQL

SQL (Structured Query Language) se divise en cinq sous-langages :

DDL – Data Definition Language

Définir la structure de la base.

Commandes : CREATE, ALTER, DROP, TRUNCATE, RENAME

DML – Data Manipulation Language

Manipuler les données dans les tables.

Commandes : INSERT, UPDATE, DELETE, MERGE

DQL – Data Query Language

Interroger et récupérer les données.

Commande principale : SELECT

DCL – Data Control Language

Gérer les droits d'accès.

Commandes : GRANT, REVOKE

TCL – Transaction Control Language

Gérer les transactions.

Commandes : BEGIN, COMMIT, ROLLBACK, SAVEPOINT

3.2 DDL – DÉFINITION DES STRUCTURES

CRÉER UNE TABLE

```
CREATE TABLE employes (  
    id          SERIAL PRIMARY KEY,  
    nom         VARCHAR(100) NOT NULL,  
    prenom      VARCHAR(100) NOT NULL,  
    email       VARCHAR(255) UNIQUE NOT NULL,  
    salaire     DECIMAL(10, 2) CHECK (salaire > 0),  
    departement_id INTEGER REFERENCES departements(id),  
    date_embauche DATE DEFAULT CURRENT_DATE,  
    actif       BOOLEAN DEFAULT TRUE  
);
```

MODIFIER UNE TABLE

```
-- Ajouter une colonne  
ALTER TABLE employes ADD COLUMN telephone VARCHAR(20);  
  
-- Modifier un type  
ALTER TABLE employes ALTER COLUMN salaire TYPE NUMERIC(12, 2);  
  
-- Supprimer une colonne  
ALTER TABLE employes DROP COLUMN telephone;  
  
-- Renommer une colonne  
ALTER TABLE employes RENAME COLUMN nom TO last_name;
```

SUPPRIMER UNE TABLE

```
DROP TABLE employes;           -- erreur si des données existent
DROP TABLE IF EXISTS employes; -- sécurisé
TRUNCATE TABLE employes;       -- vide la table, garde la structure
```

INDEX

```
-- Créer un index simple (accélère les recherches)
CREATE INDEX idx_employes_nom ON employes(nom);

-- Index unique (contrainte d'unicité)
CREATE UNIQUE INDEX idx_employes_email ON employes(email);

-- Index composite (plusieurs colonnes)
CREATE INDEX idx_employes_dept_salaire ON employes(departement_id, salaire);

-- Supprimer un index
DROP INDEX idx_employes_nom;
```

3.3 DML – MANIPULATION DES DONNÉES

INSERTION

```
-- Insertion simple
INSERT INTO employes (nom, prenom, email, salaire)
VALUES ('Diallo', 'Alpha', 'alpha@cie.gn', 5000.00);

-- Insertion multiple
INSERT INTO employes (nom, prenom, email, salaire) VALUES
    ('Bah', 'Mariama', 'm.bah@cie.gn', 4500.00),
    ('Camara', 'Ibou', 'i.camara@cie.gn', 6000.00),
    ('Sylla', 'Fatoumata', 'f.sylla@cie.gn', 4800.00);

-- Insertion depuis une requête
INSERT INTO employes_archive
SELECT * FROM employes WHERE actif = FALSE;
```

MISE À JOUR

```
-- Mise à jour simple
UPDATE employes SET salaire = 5500.00 WHERE id = 1;

-- Mise à jour multiple avec condition
UPDATE employes
SET salaire = salaire * 1.10
WHERE departement_id = 3 AND date_embauche < '2020-01-01';

-- Mise à jour avec jointure (PostgreSQL)
UPDATE employes e
SET salaire = salaire * 1.05
FROM departements d
WHERE e.departement_id = d.id AND d.nom = 'Informatique';
```

SUPPRESSION

```
-- Suppression ciblée
DELETE FROM employes WHERE id = 5;

-- Suppression avec condition
DELETE FROM employes WHERE actif = FALSE AND date_embauche < '2018-01-01';
```

3.4 SELECT – L'ART DE L'INTERROGATION

ANATOMIE D'UN SELECT

```

SELECT  [colonnes]           -- Quelles données ?
FROM    [tables]             -- D'où ?
JOIN    [table] ON [cond]    -- Avec quelles relations ?
WHERE   [condition]          -- Quels filtres ?
GROUP BY [colonnes]          -- Regrouper par ?
HAVING  [condition agrégat]  -- Filtrer les groupes ?
ORDER BY [colonnes]          -- Dans quel ordre ?
LIMIT   [n] OFFSET [m];     -- Pagination ?

```

SÉLECTIONS DE BASE

```

-- Toutes les colonnes
SELECT * FROM employes;

-- Colonnes spécifiques
SELECT nom, prenom, salaire FROM employes;

-- Alias de colonnes
SELECT nom AS "Nom de famille", salaire AS "Rémunération" FROM employes;

-- Éliminer les doublons
SELECT DISTINCT departement_id FROM employes;

-- Calcul dans la sélection
SELECT nom, salaire, salaire * 12 AS salaire_annuel FROM employes;

```

FILTRAGE AVEC WHERE

```

-- Comparaison simple
SELECT * FROM employes WHERE salaire > 5000;

-- Plusieurs conditions
SELECT * FROM employes WHERE salaire > 4000 AND departement_id = 2;

-- Intervalle
SELECT * FROM employes WHERE salaire BETWEEN 4000 AND 6000;

-- Liste de valeurs
SELECT * FROM employes WHERE departement_id IN (1, 2, 5);

-- Recherche de motif
SELECT * FROM employes WHERE nom LIKE 'D%';      -- commence par D
SELECT * FROM employes WHERE email LIKE '%@cie.gn';
SELECT * FROM employes WHERE nom ILIKE 'diallo';  -- insensible à la casse

-- Valeurs nulles
SELECT * FROM employes WHERE telephone IS NULL;
SELECT * FROM employes WHERE telephone IS NOT NULL;

```

AGRÉGATION ET GROUP BY

```

Fonctions d'agrégation :
- COUNT() : nombre de lignes
- SUM()   : somme
- AVG()   : moyenne
- MIN()   : minimum
- MAX()   : maximum

-- Compter tous les employés
SELECT COUNT(*) AS total FROM employes;

-- Statistiques de salaire par département
SELECT
    departement_id,
    COUNT(*)      AS nb_employes,
    AVG(salaire)   AS salaire_moyen,
    MIN(salaire)   AS salaire_min,
    MAX(salaire)   AS salaire_max,
    SUM(salaire)   AS masse_salariale
FROM employes

```

```

GROUP BY departement_id;

-- Filtrer les groupes avec HAVING
SELECT departement_id, AVG(salaire) AS moy
FROM employes
GROUP BY departement_id
HAVING AVG(salaire) > 5000;

```

LES JOINTURES

INNER JOIN – Intersection (lignes correspondantes des deux côtés)

```

SELECT e.nom, e.prenom, d.nom AS departement
FROM employes e
INNER JOIN departements d ON e.departement_id = d.id;

```

LEFT JOIN – Tous les enregistrements de gauche + correspondances à droite

```

SELECT e.nom, d.nom AS departement
FROM employes e
LEFT JOIN departements d ON e.departement_id = d.id;
-- Retourne aussi les employés sans département (NULL)

```

RIGHT JOIN – Tous les enregistrements de droite + correspondances à gauche

```

SELECT e.nom, d.nom AS departement
FROM employes e
RIGHT JOIN departements d ON e.departement_id = d.id;
-- Retourne aussi les départements sans employé

```

FULL OUTER JOIN – Union (tous les enregistrements des deux côtés)

```

SELECT e.nom, d.nom AS departement
FROM employes e
FULL OUTER JOIN departements d ON e.departement_id = d.id;

```

SELF JOIN – Jointure d'une table avec elle-même

```

SELECT e.nom AS employe, m.nom AS manager
FROM employes e
LEFT JOIN employes m ON e.manager_id = m.id;

```

CROSS JOIN – Produit cartésien (toutes les combinaisons)

```

SELECT e.nom, p.intitule
FROM employes e
CROSS JOIN projets p;

```

3.5 FONCTIONNALITÉS AVANCÉES SQL

SOUS-REQUÊTES (SUBQUERIES)

```

-- Dans WHERE
SELECT * FROM employes
WHERE salaire > (SELECT AVG(salaire) FROM employes);

```

```

-- Dans FROM (table dérivée)
SELECT dept_stats.departement_id, dept_stats.moy
FROM (
    SELECT departement_id, AVG(salaire) AS moy
    FROM employes
    GROUP BY departement_id
) AS dept_stats
WHERE moy > 5000;

```

```

-- Sous-requête corrélée
SELECT e.nom, e.salaire
FROM employes e
WHERE e.salaire > (
    SELECT AVG(e2.salaire)
    FROM employes e2

```

```
WHERE e2.departement_id = e.departement_id
);
```

EXPRESSIONS DE TABLE COMMUNES – CTE (WITH)

```
WITH stats_dept AS (
  SELECT
    departement_id,
    AVG(salaire) AS salaire_moyen,
    COUNT(*) AS effectif
  FROM employes
  GROUP BY departement_id
)
SELECT e.nom, e.salaire, s.salaire_moyen
FROM employes e
JOIN stats_dept s ON e.departement_id = s.departement_id
WHERE e.salaire > s.salaire_moyen;

-- CTE récursif (arborescence)
WITH RECURSIVE hierarchie AS (
  SELECT id, nom, manager_id, 0 AS niveau
  FROM employes WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.nom, e.manager_id, h.niveau + 1
  FROM employes e
  JOIN hierarchie h ON e.manager_id = h.id
)
SELECT * FROM hierarchie ORDER BY niveau;
```

FONCTIONS DE FENÊTRAGE (WINDOW FUNCTIONS)

Les fonctions de fenêtrage permettent de calculer des agrégats sans regrouper les lignes. Elles opèrent sur une "fenêtre" de données.

Syntaxe : fonction() OVER (PARTITION BY ... ORDER BY ...)

```
-- Rang dans chaque département
SELECT
  nom, departement_id, salaire,
  RANK() OVER (PARTITION BY departement_id ORDER BY salaire DESC) AS rang
FROM employes;
```

```
-- Numéro de ligne
SELECT
  nom, salaire,
  ROW_NUMBER() OVER (ORDER BY salaire DESC) AS rang_global
FROM employes;
```

```
-- Moyenne glissante (3 derniers mois)
SELECT
  mois, chiffre_affaires,
  AVG(chiffre_affaires) OVER (
    ORDER BY mois
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
  ) AS moyenne_glissante
FROM ventes;
```

```
-- Valeur précédente / suivante
SELECT
  mois, ca,
  LAG(ca) OVER (ORDER BY mois) AS ca_mois_precedent,
  LEAD(ca) OVER (ORDER BY mois) AS ca_mois_suivant
FROM ventes;
```

```
-- Cumul progressif
SELECT
  date, montant,
  SUM(montant) OVER (ORDER BY date) AS cumul
FROM transactions;
```

VUES (VIEWS)

```
-- Créer une vue
CREATE VIEW vue_employes_actifs AS
SELECT e.nom, e.prenom, e.salaire, d.nom AS departement
FROM employes e
JOIN departements d ON e.departement_id = d.id
WHERE e.actif = TRUE;

-- Utiliser la vue comme une table
SELECT * FROM vue_employes_actifs WHERE salaire > 5000;

-- Vue matérialisée (résultats mis en cache)
CREATE MATERIALIZED VIEW stats_departements AS
SELECT departement_id, AVG(salaire) AS moy, COUNT(*) AS effectif
FROM employes GROUP BY departement_id;

REFRESH MATERIALIZED VIEW stats_departements;
```

PROCÉDURES ET FONCTIONS STOCKÉES

```
-- Fonction (retourne une valeur)
CREATE OR REPLACE FUNCTION salaire_annuel(emp_id INTEGER)
RETURNS DECIMAL AS $$
DECLARE
    v_salaire DECIMAL;
BEGIN
    SELECT salaire INTO v_salaire FROM employes WHERE id = emp_id;
    RETURN v_salaire * 12;
END;
$$ LANGUAGE plpgsql;

-- Appel
SELECT salaire_annuel(3);

-- Procédure stockée (PostgreSQL 11+)
CREATE OR REPLACE PROCEDURE augmenter_salaires(pourcentage DECIMAL)
AS $$
BEGIN
    UPDATE employes SET salaire = salaire * (1 + pourcentage / 100);
    COMMIT;
END;
$$ LANGUAGE plpgsql;

CALL augmenter_salaires(10);
```

TRIGGERS

Un trigger est une procédure déclenchée automatiquement par un événement.

```
-- Créer une fonction trigger
CREATE OR REPLACE FUNCTION log_modification()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_log (table_name, action, ancien_salaire, nouveau_salaire, modifie_le)
    VALUES (TG_TABLE_NAME, TG_OP, OLD.salaire, NEW.salaire, NOW());
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Attacher le trigger
CREATE TRIGGER tr_employes_salaire
AFTER UPDATE OF salaire ON employes
FOR EACH ROW EXECUTE FUNCTION log_modification();
```

PARTIE IV – CONCEPTION ET MODÉLISATION

4.1 LE MODÈLE ENTITÉ-ASSOCIATION (EA)

La modélisation conceptuelle est l'étape préalable à toute création de base.

Elle représente les entités du domaine et leurs relations, indépendamment de toute considération technique.

CONCEPTS CLÉS

ENTITÉ

Objet du monde réel qui a une existence propre.

Exemples : Client, Produit, Commande, Employé

ATTRIBUT

Propriété décrivant une entité.

Exemples : nom, date_naissance, prix, adresse

ASSOCIATION

Lien logique entre deux ou plusieurs entités.

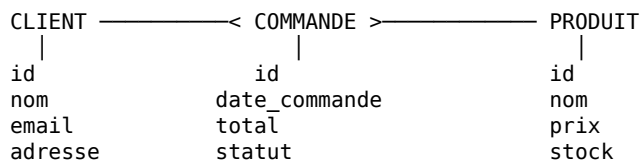
Exemples : un client PASSE une commande ; un employé APPARTIENT à un département

CARDINALITÉ

Nombre d'occurrences dans une relation.

- 1:1 (un à un)
- 1:N (un à plusieurs)
- N:M (plusieurs à plusieurs)

EXEMPLE DE SCHÉMA ENTITÉ-ASSOCIATION



Lecture : Un client passe plusieurs commandes.
Une commande contient plusieurs produits.
Un produit peut figurer dans plusieurs commandes.

4.2 PASSAGE AU MODÈLE LOGIQUE

Le modèle logique traduit le schéma EA en tables relationnelles.

RÈGLES DE TRANSFORMATION

- Règle 1 : Chaque entité devient une table.
- Règle 2 : Chaque attribut devient une colonne.
- Règle 3 : L'identifiant devient la clé primaire.
- Règle 4 : Les relations 1:N → clé étrangère du côté N.
- Règle 5 : Les relations N:M → table de jonction.

EXEMPLE RÉSULTANT

```
TABLE clients (id, nom, email, adresse)
TABLE produits (id, nom, prix, stock)
TABLE commandes (id, client_id FK→clients, date, total, statut)
TABLE commande_produits (commande_id FK→commandes,
                          produit_id FK→produits,
                          quantité, prix_unitaire)
PK composite : (commande_id, produit_id)
```

4.3 MODÉLISATION POUR LE DATAWAREHOUSE

En analytique, on utilise des schémas dénormalisés pour maximiser la performance des requêtes complexes.

SCHÉMA EN ÉTOILE (STAR SCHEMA)

Une table de faits centrale entourée de tables de dimensions.
Simple, performant, mais redondance des données.

```

FAIT_VENTES (id, date_id, client_id, produit_id, vendeur_id,
              quantite, montant, remise)
              ↑           ↑           ↑           ↑           ↑
DIM_DATE    DIM_CLIENT  DIM_PRODUIT DIM_VENDEUR

```

SCHÉMA EN FLOCON (SNOWFLAKE SCHEMA)

Extension du schéma en étoile où les dimensions sont normalisées.
Moins de redondance, mais jointures plus complexes.

PARTIE V – OPTIMISATION ET PERFORMANCE SQL

5.1 COMPRENDRE LE PLAN D'EXÉCUTION

Avant d'optimiser, il faut comprendre ce que fait le SGBD.
La commande EXPLAIN (ou EXPLAIN ANALYZE) révèle le plan d'exécution.

```

EXPLAIN ANALYZE
SELECT e.nom, d.nom
FROM employes e JOIN departements d ON e.departement_id = d.id
WHERE e.salaire > 5000;

```

ÉLÉMENTS À ANALYSER

- Seq Scan : lecture séquentielle (lente sur grandes tables)
- Index Scan : utilisation d'un index (rapide)
- Hash Join : jointure par hachage (efficace sur grands volumes)
- Nested Loop : boucle imbriquée (efficace si petits ensembles)
- Sort : tri (coûteux sans index adapté)
- Cost : coût estimé (unité arbitraire)
- Actual Time : temps réel mesuré

5.2 LES INDEX EN PROFONDEUR

TYPES D'INDEX

B-TREE (défaut)

Structure arborescente équilibrée. Adapté aux comparaisons (=, <, >, BETWEEN).

HASH

Adapté uniquement à l'égalité exacte (=). Plus rapide que B-tree dans ce cas.

GIN (Generalized Inverted Index)

Adapté aux recherches full-text, tableaux, JSONB.

GIST (Generalized Search Tree)

Adapté aux données géospatiales, intervalles, formes géométriques.

BRIN (Block Range Index)

Très compact. Adapté aux grandes tables avec données ordonnées naturellement (ex : logs chronologiques, séries temporelles).

QUAND CRÉER UN INDEX ?

- ✓ Colonnes fréquemment utilisées dans WHERE
- ✓ Colonnes utilisées dans JOIN ON
- ✓ Colonnes utilisées dans ORDER BY
- ✓ Colonnes à forte cardinalité (peu de doublons)

QUAND NE PAS CRÉER D'INDEX ?

- ✗ Tables très petites (scan séquentiel plus rapide)
- ✗ Colonnes à très faible cardinalité (ex : booléen, statut)
- ✗ Tables avec beaucoup d'écritures (les index ralentissent les INSERT/UPDATE)

5.3 BONNES PRATIQUES D'OPTIMISATION

-
- [1] ÉVITER SELECT *
Sélectionner uniquement les colonnes nécessaires.
SELECT * empêche les index de couvrir la requête.
 - [2] FILTRER TÔT
Placer les conditions les plus restrictives en premier.
Réduire le nombre de lignes avant les jointures.
 - [3] UTILISER LES INDEX EFFICACEMENT
Ne pas appliquer de fonction sur une colonne indexée.
✗ WHERE YEAR(date_embauche) = 2020
✓ WHERE date_embauche BETWEEN '2020-01-01' AND '2020-12-31'
 - [4] LIMITER LES SOUS-REQUÊTES CORRÉLÉES
Elles s'exécutent une fois par ligne. Très coûteuses.
Préférer les JOIN ou les CTE.
 - [5] PAGINATION EFFICACE
✗ SELECT * FROM logs LIMIT 20 OFFSET 100000 (charge 100020 lignes)
✓ SELECT * FROM logs WHERE id > 100000 ORDER BY id LIMIT 20
 - [6] ANALYSER LES STATISTIQUES
ANALYZE employes; -- met à jour les statistiques du planificateur
VACUUM ANALYZE; -- libère l'espace et met à jour les stats (PostgreSQL)
 - [7] PARTITIONNEMENT
Diviser une grande table en partitions plus petites.

CREATE TABLE ventes (
 id SERIAL, date DATE, montant DECIMAL
) PARTITION BY RANGE (date);

CREATE TABLE ventes_2024 PARTITION OF ventes
 FOR VALUES FROM ('2024-01-01') TO ('2025-01-01');

PARTIE VI – SGBD RELATIONNELS MAJEURS

6.1 POSTGRESQL

PRÉSENTATION

Open source, gratuit, le plus avancé techniquement.
Soutenu par une communauté mondiale active depuis 1996.
Conforme aux standards SQL, extensible, robuste.

POINTS FORTS

- Support natif de JSON/JSONB (hybride SQL/NoSQL)
- Extensions puissantes (PostGIS pour géospatial, TimescaleDB, etc.)
- Fonctions de fenêtrage complètes
- CTE récursifs, LATERAL JOIN, upsert (INSERT ON CONFLICT)
- Réplication logique et physique
- Héritage de tables, types personnalisés

UTILISATION TYPIQUE

Applications web complexes, analytique, SaaS, data engineering.
Préférées par les développeurs et data engineers sérieux.

6.2 MYSQL / MARIADB

PRÉSENTATION

MySQL : le plus populaire historiquement (racheté par Oracle en 2010).
MariaDB : fork communautaire de MySQL, plus ouvert et souvent plus rapide.

POINTS FORTS

- Très répandu, large écosystème
- Excellent pour le web (LAMP stack : Linux, Apache, MySQL, PHP)
- Facile à installer et configurer
- Bonne performance en lecture

LIMITES

- Moins de fonctionnalités avancées que PostgreSQL
- Conformité SQL partielle
- MariaDB comble progressivement ces lacunes

MOTEURS DE STOCKAGE (MySQL)

- InnoDB : transactions ACID, clés étrangères (défaut recommandé)
- MyISAM : plus rapide en lecture pure, mais sans transactions
- MEMORY : données en RAM, très rapide, pas de persistance

6.3 ORACLE DATABASE

PRÉSENTATION

Le SGBD d'entreprise le plus puissant et le plus coûteux.
Dominant dans les grandes entreprises, banques, gouvernements.

POINTS FORTS

- Performances extrêmes sur grands volumes
- Haute disponibilité (Oracle RAC : Real Application Clusters)
- Sécurité enterprise-grade
- Outils de migration, DBA et monitoring avancés
- PL/SQL : langage procédural très riche

LIMITES

- Licence très coûteuse
- Complexité d'administration
- Moins adapté aux startups et PME

6.4 MICROSOFT SQL SERVER

PRÉSENTATION

SGBD de Microsoft, très intégré à l'écosystème Windows / Azure.
Populaire en entreprise, surtout dans les environnements Microsoft.

POINTS FORTS

- Intégration native avec .NET, Azure, Power BI
- SQL Server Integration Services (SSIS) pour ETL
- SQL Server Analysis Services (SSAS) pour OLAP
- SQL Server Reporting Services (SSRS) pour rapports
- T-SQL : variante de SQL avec extensions Microsoft

6.5 SQLITE

PRÉSENTATION

Base de données embarquée, sans serveur, stockée dans un seul fichier.
La plus déployée au monde (smartphones, navigateurs, appareils IoT).

UTILISATION

- Applications mobiles (iOS, Android)
- Prototypage et développement local
- Fichiers de configuration structurés
- Tests unitaires

LIMITES

- Non adapté aux accès concurrents massifs
- Pas de serveur → pas de gestion multi-utilisateurs avancée

PARTIE VII – INTRODUCTION AU NOSQL

7.1 POURQUOI LE NOSQL EST APPARU

Au milieu des années 2000, les géants du web ont rencontré des limites impossibles à dépasser avec les SGBD relationnels classiques :

PROBLÈMES RENCONTRÉS

[1] VOLUME

Facebook stockait des milliards de profils et interactions.
Les bases relationnelles ne montaient pas à cette échelle.

[2] VÉLOCITÉ

Twitter devait ingérer des millions de tweets par heure.
Les contraintes ACID ralentissaient les écritures.

[3] VARIÉTÉ

Les données web sont semi-structurées ou non structurées.
Imposer un schéma fixe à des données hétérogènes est contre-productif.

[4] SCALABILITÉ HORIZONTALE

Les bases SQL scalent verticalement (serveur plus puissant).
Le NoSQL permet de scaler horizontalement (plus de serveurs).

RÉPONSES FONDATRICES

- 2006 : Google publie "Bigtable: A Distributed Storage System"
- 2007 : Amazon publie "Dynamo: Amazon's Highly Available Key-Value Store"
- 2009 : terme "NoSQL" popularisé lors d'un meetup à San Francisco

7.2 CARACTÉRISTIQUES COMMUNES DU NOSQL

SCHEMA FLEXIBLE (SCHEMA-LESS)

Pas de schéma rigide imposé à l'avance.
Chaque document / enregistrement peut avoir des champs différents.
Avantage : agilité de développement, évolution du modèle sans migration.
Risque : incohérences si non maîtrisé.

SCALABILITÉ HORIZONTALE

Les données sont distribuées sur de nombreux serveurs (nœuds).
Ajouter un nœud augmente la capacité sans arrêter le système.
Concept clé : le sharding (partitionnement des données entre nœuds).

HAUTE DISPONIBILITÉ

Réplication des données sur plusieurs nœuds.
En cas de panne d'un nœud, les autres prennent le relais.
Le système reste disponible même en cas de défaillance partielle.

MODÈLE DE COHÉRENCE ASSOUPLE

Le NoSQL accepte souvent une cohérence éventuelle (eventual consistency).
Les données propagées entre nœuds peuvent être temporairement différentes.
Un compromis accepté pour gagner en performance et disponibilité.

7.3 FAMILLES DE BASES NOSQL

FAMILLE	MODÈLE	EXEMPLES
Clé-Valeur	Dictionnaire	Redis, DynamoDB, Memcached
Document	JSON/BSON	MongoDB, CouchDB, Firestore
Colonne Large	Tables colonnes	Cassandra, HBase, BigTable
Graphe	Nœuds & arêtes	Neo4j, Neptune, ArangoDB

PARTIE VIII – NOSQL CLÉ-VALEUR

8.1 PRINCIPE

Le modèle le plus simple du NoSQL.
Chaque entrée est une paire : une clé unique → une valeur associée.
La valeur est opaque pour la base (chaîne, nombre, objet sérialisé...).

Structure : { "clé" → "valeur" }

Exemple :

```
"session:user:12345" → '{"user_id":12345,"nom":"Alpha","token":"xyz"}'
"cache:produit:89" → '{"prix":25.99,"stock":120}'
"compteur:vues:home" → "47832"
```

Opérations de base :

- GET(clé) : récupérer une valeur
- SET(clé, valeur) : stocker une valeur
- DEL(clé) : supprimer une entrée
- EXISTS(clé) : vérifier si une clé existe
- EXPIRE(clé, ttl) : définir une durée de vie

8.2 REDIS

PRÉSENTATION

Remote Dictionary Server. Base de données en mémoire, ultra-rapide.
Open source, écrit en C, latence sub-milliseconde.
Le plus populaire de sa catégorie.

STRUCTURES DE DONNÉES SUPPORTÉES

- String : valeur simple (texte, nombre, binaire)
- List : liste ordonnée (file d'attente, historique)
- Set : ensemble de valeurs uniques
- Sorted Set : ensemble ordonné par score (classements)
- Hash : dictionnaire de champs (objet structuré)
- HyperLogLog : estimation de cardinalité
- Bitmap : opérations bit à bit
- Stream : flux d'événements (comme Kafka léger)
- Geospatial : coordonnées GPS avec recherche de proximité

COMMANDES REDIS ESSENTIELLES

```
-- String
SET utilisateur:1:nom "Alpha Diallo"
GET utilisateur:1:nom
INCR compteur:visites
SETEX session:abc123 3600 '{"user_id":1}' -- expire dans 3600s

-- List (file d'attente)
LPUSH taches "envoyer_email"
RPUSH taches "generer_rapport"
LRANGE taches 0 -1 -- toutes les tâches
RPOP taches -- dépiler

-- Sorted Set (classement)
ZADD classement 4850 "Alpha"
```

```

ZADD classement 5200 "Mariama"
ZADD classement 3900 "Ibou"
ZREVRANGE classement 0 2 WITHSCORES -- top 3

-- Hash (objet)
HSET produit:42 nom "Téléphone" prix 350 stock 15
HGET produit:42 prix
HGETALL produit:42

-- TTL (expiration)
EXPIRE session:abc123 1800 -- expire dans 30 min
TTL session:abc123 -- temps restant

```

CAS D'USAGE REDIS

- Cache applicatif (réduire les appels base de données)
- Gestion de sessions utilisateurs
- Files d'attente de tâches (job queue)
- Classements en temps réel (leaderboards)
- Pub/Sub (messagerie entre services)
- Compteurs et statistiques temps réel
- Rate limiting (limitation de débit API)

PERSISTANCE REDIS

Par défaut en mémoire → données perdues si redémarrage.
Options de persistance :

- RDB (snapshot) : sauvegarde périodique sur disque
- AOF (Append Only File) : journalisation de chaque commande
- Hybride RDB + AOF : recommandé en production

8.3 AMAZON DYNAMODB

PRÉSENTATION

Service NoSQL entièrement géré par AWS (Amazon Web Services).
Scalabilité automatique, haute disponibilité garantie.
Utilisé à grande échelle par Amazon pour son e-commerce.

CONCEPTS CLÉS

- Table : collection d'items
- Item : enregistrement (équivalent d'un document)
- Attribut : champ d'un item
- Partition Key : clé de partitionnement (obligatoire)
- Sort Key : clé de tri (optionnelle, combinée à la partition key)

MODÈLES D'ACCÈS

DynamoDB est optimisé pour des patterns d'accès définis à l'avance.
La conception doit partir des requêtes, pas des entités.

CAS D'USAGE DYNAMODB

- Applications mobile/web à fort trafic
- Paniers d'achat, sessions, préférences
- Jeux en ligne (scores, états de partie)
- IoT (données capteurs)
- Tout système nécessitant une latence < 10ms à n'importe quelle échelle

PARTIE IX – NOSQL DOCUMENT

9.1 PRINCIPE

Les bases document stockent des données sous forme de documents semi-structurés, généralement en format JSON, BSON ou XML.

Un document est une unité autonome contenant toutes les informations

liées à une entité, y compris les données imbriquées.

Structure d'un document MongoDB :

```
{
  "_id": "ObjectId('64f3alb2c9e1234567890abc')",
  "nom": "Diallo",
  "prenom": "Alpha",
  "email": "alpha@cie.gn",
  "adresse": {
    "rue": "Avenue de la République",
    "ville": "Conakry",
    "pays": "Guinée"
  },
  "commandes": [
    { "id": 1, "produit": "Ordinateur", "montant": 1200 },
    { "id": 2, "produit": "Souris", "montant": 25 }
  ],
  "date_inscription": "2024-03-15",
  "actif": true
}
```

Avantage clé : toutes les données d'un client dans un seul document.
Pas de jointure nécessaire pour récupérer un profil complet.

9.2 MONGODB

PRÉSENTATION

La base de données document la plus populaire au monde.
Open source, créée en 2009 par 10gen (devenu MongoDB Inc.).
Stocke les données en BSON (Binary JSON).

TERMINOLOGIE

SQL	→	MongoDB
Base	→	Database
Table	→	Collection
Ligne	→	Document
Colonne	→	Champ (Field)
JOIN	→	\$lookup / embedding
PRIMARY KEY	→	_id (ObjectId automatique)

OPÉRATIONS CRUD MONGODB

```
// Insérer un document
db.clients.insertOne({
  nom: "Alpha",
  email: "alpha@cie.gn",
  ville: "Conakry"
});

// Insérer plusieurs documents
db.clients.insertMany([
  { nom: "Mariama", email: "m@cie.gn", ville: "Kindia" },
  { nom: "Ibou", email: "i@cie.gn", ville: "Labé" }
]);

// Trouver tous les documents
db.clients.find();

// Filtre simple
db.clients.find({ ville: "Conakry" });

// Filtre avec opérateurs
db.clients.find({ age: { $gt: 25, $lt: 50 } });
db.clients.find({ ville: { $in: ["Conakry", "Kindia"] } });

// Projection (choisir les champs retournés)
db.clients.find({ ville: "Conakry" }, { nom: 1, email: 1, _id: 0 });

// Tri, limite, saut
```

```

db.clients.find().sort({ nom: 1 }).limit(10).skip(20);

// Mettre à jour un document
db.clients.updateOne(
  { email: "alpha@cie.gn" },
  { $set: { ville: "Dubréka", actif: true } }
);

// Mettre à jour plusieurs documents
db.clients.updateMany(
  { ville: "Kindia" },
  { $inc: { points: 100 } } // incrémenter de 100
);

// Supprimer
db.clients.deleteOne({ email: "alpha@cie.gn" });
db.clients.deleteMany({ actif: false });

```

AGRÉGATION MONGODB

L'Aggregation Pipeline est l'équivalent MongoDB des requêtes complexes SQL.

```

// Chiffre d'affaires par ville (équivalent GROUP BY)
db.commandes.aggregate([
  { $match: { statut: "livree" } }, // WHERE
  { $group: {
    _id: "$ville",
    total: { $sum: "$montant" },
    nb: { $count: {} }
  } },
  { $sort: { total: -1 } }, // ORDER BY
  { $limit: 5 } // TOP 5
]);

// Jointure ($lookup = LEFT JOIN)
db.commandes.aggregate([
  { $lookup: {
    from: "clients",
    localField: "client_id",
    foreignField: "_id",
    as: "client_info"
  } },
  { $unwind: "$client_info" },
  { $project: {
    montant: 1,
    "client_info.nom": 1
  } }
]);

```

INDEX MONGODB

```

// Index simple
db.clients.createIndex({ email: 1 });

// Index unique
db.clients.createIndex({ email: 1 }, { unique: true });

// Index composé
db.clients.createIndex({ ville: 1, nom: 1 });

// Index texte (full-text search)
db.articles.createIndex({ titre: "text", contenu: "text" });
db.articles.find({ $text: { $search: "base de données" } });

// Index géospatial
db.magasins.createIndex({ location: "2dsphere" });

```

MODÉLISATION EN MONGODB

Deux stratégies principales :

EMBEDDING (Intégration)

- Inclure les données liées dans le même document.
- ✓ Une seule requête pour lire tout le document
- ✓ Idéal pour les relations 1:1 et 1:N avec petit N
- ✗ Document trop lourd si les données imbriquées sont nombreuses

REFERENCING (Référence)

Stocker l'_id de l'autre document (comme une clé étrangère).

- ✓ Données partagées non dupliquées
- ✓ Documents plus légers
- ✗ Nécessite des requêtes supplémentaires (\$lookup)

RÈGLE PRATIQUE :

Données accédées ensemble → embed

Données accédées séparément ou partagées → reference

PARTIE X – NOSQL COLONNE LARGE

10.1 PRINCIPE

Les bases orientées colonnes (wide-column) organisent les données par colonnes plutôt que par lignes. Chaque ligne peut avoir un ensemble différent de colonnes.

DIFFÉRENCE AVEC LE MODÈLE RELATIONNEL

SQL (orienté lignes) :

id	nom	email	ville
1	Alpha	a@..	Cky

← toutes les colonnes stockées ensemble

NoSQL Colonne (orienté colonnes) :

Colonne "nom" : [1→Alpha, 2→Mariama, 3→Ibou]

Colonne "email" : [1→a@.., 2→m@.., 3→i@..] ← chaque colonne stockée séparément

Colonne "ville" : [1→Cky, 2→Kindia, 3→Labé]

AVANTAGE ANALYTIQUE :

Lire une seule colonne = lire uniquement cette colonne sur disque.

Idéal pour les requêtes analytiques qui agrègent quelques colonnes sur des millions de lignes.

10.2 APACHE CASSANDRA

PRÉSENTATION

Créée par Facebook, open source depuis 2008.

Conçue pour une disponibilité maximale et une scalabilité linéaire.

Aucun point de défaillance unique (architecture peer-to-peer).

Utilisée par Netflix, Instagram, Uber, Apple.

ARCHITECTURE

- Cluster : ensemble de nœuds
- Nœud : serveur Cassandra
- Datacenter : groupe logique de nœuds
- Anneau : les nœuds forment un anneau de hachage
- Réplication : les données sont copiées sur N nœuds (replication factor)

MODÈLE DE DONNÉES

CQL (Cassandra Query Language) ressemble à SQL mais avec des restrictions.

```
-- Créer un keyspace (= base de données)
CREATE KEYSPACE boutique
WITH replication = {
  'class': 'SimpleStrategy',
  'replication_factor': 3
}
```

```

};

-- Créer une table
CREATE TABLE boutique.commandes (
  client_id  UUID,
  date       TIMESTAMP,
  commande_id  UUID,
  montant    DECIMAL,
  statut     TEXT,
  PRIMARY KEY ((client_id), date, commande_id)
);
-- Partition key : client_id (toutes les commandes d'un client ensemble)
-- Clustering columns : date, commande_id (tri dans la partition)

-- Insérer
INSERT INTO boutique.commandes (client_id, date, commande_id, montant)
VALUES (uuid(), toTimestamp(now()), uuid(), 150.00);

-- Lire (les requêtes DOIVENT utiliser la partition key)
SELECT * FROM boutique.commandes WHERE client_id = ?;

```

RÈGLE FONDAMENTALE CASSANDRA

Concevoir les tables en fonction des requêtes, pas des entités.
 Une requête = une table. Il n'y a pas de jointures.
 La dénormalisation est une pratique normale et attendue.

TUNABLE CONSISTENCY

On règle le niveau de cohérence selon le besoin :

- ONE : réponse du premier nœud (le plus rapide, moins fiable)
- QUORUM : majorité des nœuds doit confirmer (équilibre)
- ALL : tous les nœuds doivent confirmer (le plus fiable, le plus lent)

10.3 APACHE HBASE ET GOOGLE BIGTABLE

HBASE

Implémentation open source de Bigtable (Google).
 Intégré à l'écosystème Hadoop (HDFS, MapReduce).
 Adapté aux très grands volumes de données (pétaoctets).
 Utilisé pour les logs, les données IoT, l'historique de transactions.

BIGTABLE

Service géré par Google Cloud.
 Alimente en interne Search, Maps, Gmail, Analytics.
 Latence constante même à des milliards de lignes.

PARTIE XI – NOSQL GRAPHE

11.1 PRINCIPE

Les bases de données graphe modélisent les données comme un réseau :
 des nœuds (entités) reliés par des arêtes (relations).

COMPOSANTS

- Nœuds (Nodes) : les entités (Personne, Produit, Ville...)
- Arêtes (Edges) : les relations entre entités (AIME, CONNAÎT, ACHÈTE...)
- Propriétés : attributs attachés aux nœuds et aux arêtes

EXEMPLE DE GRAPHE SOCIAL

```

(Alpha) —[CONNAÎT]→ (Mariama)
  |                   |
[AIME]                [SUIT]

```

↓
(Alpha) ← cycle possible !

POURQUOI LES GRAPHS ?

Les relations sont des données de première classe.
Les jointures SQL sur des relations multi-niveaux sont exponentiellement lentes.
Dans un graphe, traverser des relations est une opération native et rapide.

CAS D'USAGE IDÉAUX

- Réseaux sociaux (amis d'amis, recommandations)
- Détection de fraude (chaînes de transactions suspectes)
- Moteurs de recommandation
- Graphes de connaissance (knowledge graphs)
- Gestion de dépendances (supply chain, infrastructure)
- Cybersécurité (cartographie des attaques)

11.2 NE04J

PRÉSENTATION

La base de données graphe la plus populaire.
Utilise le langage de requête Cypher.
Open source (Community Edition) et Enterprise.
Utilisée par eBay, Airbus, NASA, gouvernements.

LANGUAGE CYPHER

Cypher utilise une syntaxe visuelle inspirée des graphes.
Les nœuds sont représentés par des parenthèses.
Les relations par des flèches.

```
// Créer des nœuds
CREATE (a:Personne {nom: "Alpha", age: 30})
CREATE (m:Personne {nom: "Mariama", age: 27})
CREATE (j:Musique {genre: "Jazz"})

// Créer des relations
CREATE (a)-[:CONNAÎT]->(m)
CREATE (a)-[:AIME    ]->(j)
CREATE (m)-[:AIME    ]->(j)

// Trouver tous les amis d'Alpha
MATCH (a:Personne {nom: "Alpha"})-[:CONNAÎT]->(ami)
RETURN ami.nom;

// Trouver les amis d'amis (2 niveaux)
MATCH (a:Personne {nom: "Alpha"})-[:CONNAÎT*2]->(ami2)
RETURN DISTINCT ami2.nom;

// Chemin le plus court entre deux personnes
MATCH p = shortestPath(
  (a:Personne {nom: "Alpha"})-[:CONNAÎT*]- (cible:Personne {nom: "Ibou"})
)
RETURN p, length(p) AS distance;

// Recommandation : que lisent les amis d'Alpha qu'il ne lit pas déjà ?
MATCH (a:Personne {nom: "Alpha"})-[:CONNAÎT]->(ami)-[:AIME]->(item)
WHERE NOT (a)-[:AIME]->(item)
RETURN item.genre, COUNT(*) AS popularite
ORDER BY popularite DESC;

// Détection de fraude : transactions en chaîne suspectes
MATCH (c:Compte)-[:TRANSFERT*3..5]->(suspect:Compte)
WHERE c.pays <> suspect.pays
RETURN c, suspect;
```

PARTIE XII – SGBD MULTI-MODÈLES ET NEWSQL

12.1 NEWSQL

Le NewSQL tente de combiner le meilleur des deux mondes :

- La scalabilité horizontale du NoSQL
- Les garanties ACID et le SQL du modèle relationnel

REPRÉSENTANTS MAJEURS

GOOGLE SPANNER

Base de données distribuée globalement par Google.
Garantit la cohérence forte (ACID) à l'échelle planétaire.
Utilise des horloges atomiques et GPS pour synchroniser les nœuds.
Utilisé par Google Finance, AdWords, Play Store.

COCKROACHDB

Open source, inspiré de Spanner.
SQL standard (compatible PostgreSQL), distribué, survivant aux pannes.
Réplication automatique entre datacenters.

TIDB

MySQL-compatible, distribué, open source.
Populaire en Asie (PingCAP, entreprise chinoise).

YUGABYTEDB

Compatible PostgreSQL et Cassandra.
ACID + scalabilité horizontale + multi-région.

12.2 SGBD MULTI-MODÈLES

ARANGODB

Supporte document, clé-valeur et graphe dans une seule base.
Langage de requête : AQL (ArangoDB Query Language).
Open source, performant pour les graphes et documents mixtes.

AZURE COSMOSDB (MICROSOFT)

Service cloud multi-modèle entièrement géré.
Supporte plusieurs API : SQL, MongoDB, Cassandra, Gremlin (graphe), Table.
Disponibilité mondiale garantie avec SLA.
Cohérence configurable (5 niveaux).

FAUNADB

Base serverless multi-modèle avec cohérence forte globale.
API GraphQL native. Adapté aux applications JAMstack.

12.3 BASES SPÉCIALISÉES

SÉRIES TEMPORELLES

Optimisées pour les données horodatées séquentielles.

InfluxDB : métriques d'infrastructure, IoT
TimescaleDB : extension PostgreSQL pour séries temporelles
Prometheus : métriques pour systèmes cloud-native

RECHERCHE FULL-TEXT

Optimisées pour la recherche textuelle complexe.

Elasticsearch : recherche et analytique en temps réel
OpenSearch : fork open source d'Elasticsearch
Solr : moteur Apache, très configurable

VECTORIELLES

Nouvelles bases adaptées à l'IA et aux embeddings.
Stockent des vecteurs de haute dimension pour la recherche sémantique.

Pinecone, Weaviate, Milvus, Qdrant, pgvector (extension PostgreSQL)
Utilisées pour : RAG (Retrieval Augmented Generation), recherche d'images,
systèmes de recommandation par similarité.

PARTIE XIII – TRANSACTIONS, ACID ET CAP

13.1 LES PROPRIÉTÉS ACID

ACID est l'acronyme des quatre propriétés qui garantissent la fiabilité des transactions dans un SGBD.

[A] ATOMICITÉ (Atomicity)

Une transaction est tout ou rien.
Si une opération échoue, toutes les opérations précédentes sont annulées.

Exemple : virement bancaire
Débit du compte A : -500€
Crédit du compte B : +500€
→ Si le crédit échoue, le débit est annulé. Pas d'argent perdu.

[C] COHÉRENCE (Consistency)

Une transaction amène la base d'un état valide à un autre état valide.
Toutes les contraintes d'intégrité restent respectées.

[I] ISOLATION (Isolation)

Les transactions concurrentes ne s'interfèrent pas.
Chaque transaction voit les données comme si elle était la seule.

Niveaux d'isolation (du moins au plus strict) :

- READ UNCOMMITTED : voit les modifications non validées (lecture sale)
- READ COMMITTED : voit uniquement les modifications validées
- REPEATABLE READ : les lectures répétées donnent le même résultat
- SERIALIZABLE : isolation totale, comme si les transactions étaient séquentielles

[D] DURABILITÉ (Durability)

Une transaction validée (COMMIT) est définitivement enregistrée.
Même après une panne, les données sont préservées.
Garanti par les journaux de transactions (WAL).

EXEMPLE DE TRANSACTION SQL

```
BEGIN;  
  UPDATE comptes SET solde = solde - 500 WHERE id = 1; -- Débit  
  UPDATE comptes SET solde = solde + 500 WHERE id = 2; -- Crédit  
  -- Vérification  
  SELECT solde FROM comptes WHERE id = 1;  
COMMIT; -- Valider si tout est ok  
-- ou ROLLBACK; pour annuler
```

13.2 LE THÉORÈME CAP

Formulé par Eric Brewer en 2000, le théorème CAP stipule qu'un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes:

[C] COHÉRENCE (Consistency)

Tous les nœuds voient les mêmes données au même moment.
Toute lecture retourne la valeur la plus récente.

[A] DISPONIBILITÉ (Availability)

Toute requête reçoit une réponse (pas forcément la plus récente).
Le système répond toujours, même en cas de panne partielle.

[P] TOLÉRANCE AUX PARTITIONS (Partition Tolerance)

Le système continue de fonctionner même si des nœuds ne communiquent plus.
(Partition réseau = coupure de communication entre nœuds)

POSITIONNEMENT DES SGBD

CA (Cohérence + Disponibilité) :

- Systèmes non distribués : PostgreSQL, MySQL, Oracle
- Sacrifient la tolérance aux partitions (un seul nœud ou réseau fiable)

CP (Cohérence + Tolérance aux partitions) :

- HBase, MongoDB (en mode fort), Zookeeper
- En cas de partition : refusent les requêtes plutôt que retourner des données obsolètes

AP (Disponibilité + Tolérance aux partitions) :

- Cassandra, CouchDB, DynamoDB (en mode standard)
- En cas de partition : répondent avec des données potentiellement obsolètes

13.3 COHÉRENCE ÉVENTUELLE

La cohérence éventuelle (eventual consistency) est le modèle adopté par la plupart des systèmes AP NoSQL.

PRINCIPE :

Si aucune mise à jour n'est effectuée, toutes les répliques convergeront éventuellement vers la même valeur.

CONCRÈTEMENT :

- Vous modifiez votre photo de profil Facebook.
- Pendant quelques secondes, certains utilisateurs voient l'ancienne photo.
- Quelques secondes plus tard, tout le monde voit la nouvelle.
- C'est acceptable pour cette donnée.

QUAND C'EST INACCEPTABLE :

- Solde bancaire (cohérence forte requise)
- Réservation de place (pas de double réservation)
- Stock critique (pas de survente)

MODÈLES INTERMÉDIAIRES (BASE)

- Basically Available : disponibilité prioritaire
- Soft state : l'état peut changer sans action explicite
- Eventual consistency : cohérence garantie à terme

PARTIE XIV – SQL vs NoSQL : CHOISIR LE BON OUTIL

14.1 TABLEAU COMPARATIF

CRITÈRE	SQL (Relationnel)	NoSQL
Schéma	Fixe, défini à l'avance	Flexible, schema-less
Données	Structurées	Semi/non structurées
Scalabilité	Verticale (scale-up)	Horizontale (scale-out)
Transactions	ACID complet	Souvent BASE / ACID limité
Requêtes complexes	Excellent (JOIN, agrégat)	Variable (dépend du type)
Jointures	Natives et optimisées	Limitées ou absentes
Cohérence	Forte	Éventuelle (souvent)
Courbe d'apprent.	SQL bien connu	Variable par technologie
Flexibilité modèle	Faible (migrations lentes)	Élevée
Maturité	50+ ans	15-20 ans
Écosystème	Très large, standardisé	Fragmenté par produit

14.2 QUAND UTILISER SQL ?

- ✓ Données avec des relations complexes entre entités
- ✓ Transactions financières et opérations critiques (ACID requis)
- ✓ Applications avec un schéma stable et bien défini
- ✓ Requêtes analytiques complexes (reporting, BI)
- ✓ Équipes habituées à SQL, standards importants
- ✓ Conformité réglementaire forte (RGPD, normes bancaires)

EXEMPLES DE CAS SQL

- Système bancaire, ERP, CRM
- Application de gestion (RH, comptabilité, stock)
- Datawarehouse et reporting (avec PostgreSQL, Redshift, BigQuery)
- Application e-commerce classique

14.3 QUAND UTILISER NoSQL ?

- ✓ Volumes très importants avec scalabilité horizontale requise
- ✓ Données hétérogènes ou dont le schéma évolue rapidement
- ✓ Faible latence requise à grande échelle (sous-milliseconde)
- ✓ Cas d'usage spécialisés (graphes, cache, séries temporelles)
- ✓ Architecture microservices avec data ownership par service
- ✓ Prototypage rapide sans contrainte de schéma

EXEMPLES DE CAS NoSQL

- Réseaux sociaux, messagerie, feed d'actualités → Document / Colonne
- Cache et sessions → Redis (Clé-Valeur)
- Recommandations, réseaux d'amis → Neo4j (Graphe)
- Logs et métriques → Cassandra / InfluxDB
- Catalogue produit e-commerce → MongoDB
- IoT, capteurs temps réel → Cassandra / TimescaleDB

14.4 LA POLYGLOT PERSISTENCE

En architecture moderne, il n'y a pas de choix unique.

La persistance polyglotte consiste à utiliser plusieurs types de SGBD, chacun optimisé pour sa partie du problème.

EXEMPLE D'ARCHITECTURE E-COMMERCE

PostgreSQL	→ Commandes, paiements (ACID requis)
MongoDB	→ Catalogue produits (schéma flexible, attributs variables)
Redis	→ Sessions, panier, cache des pages
Elasticsearch	→ Recherche produits (full-text, facettes)
Neo4j	→ Moteur de recommandation (produits souvent achetés ensemble)
Cassandra	→ Historique de navigation (volume massif, écriture rapide)

MISE EN GARDE :

La polyglot persistence augmente la complexité opérationnelle.

Chaque base à maintenir, sauvegarder, monitorer.

Ne pas multiplier les outils sans justification technique réelle.

PARTIE XV – BASE DE DONNÉES EN PRODUCTION

15.1 SÉCURITÉ

AUTHENTIFICATION ET AUTORISATION

```
-- Créer un utilisateur (PostgreSQL)
CREATE USER app_user WITH PASSWORD 'motdepasse_fort_123!';

-- Accorder des droits minimaux (principe du moindre privilège)
```

```
GRANT SELECT, INSERT, UPDATE ON TABLE commandes TO app_user;  
GRANT SELECT ON TABLE produits TO app_user;
```

```
-- Révoquer des droits  
REVOKE DELETE ON TABLE commandes FROM app_user;
```

PROTECTION DES DONNÉES

- Chiffrement au repos (TDE : Transparent Data Encryption)
- Chiffrement en transit (SSL/TLS)
- Masquage des données sensibles (anonymisation)
- Hachage des mots de passe (bcrypt, argon2)

INJECTION SQL

L'injection SQL est l'attaque la plus répandue sur les bases de données.

VULNÉRABLE (ne jamais faire) :

```
query = "SELECT * FROM users WHERE email = '" + user_input + "'";  
-- Si input = "'; DROP TABLE users; --" → catastrophe
```

SÉCURISÉ (toujours utiliser des requêtes paramétrées) :

```
query = "SELECT * FROM users WHERE email = $1"  
execute(query, [user_input]) -- le paramètre est échappé automatiquement
```

15.2 SAUVEGARDE ET RESTAURATION

STRATÉGIES DE SAUVEGARDE

SAUVEGARDE COMPLÈTE (Full Backup)

Copie intégrale de toute la base. Simple mais volumineuse et lente.

SAUVEGARDE INCRÉMENTALE

Sauvegarde uniquement les changements depuis la dernière sauvegarde.

Rapide mais restauration complexe (appliquer chaque incrément).

SAUVEGARDE DIFFÉRENTIELLE

Sauvegarde les changements depuis la dernière sauvegarde complète.

Compromis entre complète et incrémentale.

POINT-IN-TIME RECOVERY (PITR)

Restauration à un instant précis grâce aux journaux WAL.

Exemple : "restaurer la base telle qu'elle était hier à 14h37"

COMMANDES POSTGRESQL

```
# Sauvegarder une base  
pg_dump -U postgres -Fc ma_base > ma_base_backup.dump  
  
# Restaurer  
pg_restore -U postgres -d ma_base_restaurée ma_base_backup.dump  
  
# Sauvegarde de tout le cluster  
pg_basebackup -D /backup/base -Ft -z -P
```

15.3 HAUTE DISPONIBILITÉ ET RÉPLICATION

RÉPLICATION PRIMAIRE / RÉPLIQUE

Un serveur principal (primary/master) reçoit les écritures.

Un ou plusieurs serveurs répliques (replicas/slaves) reçoivent une copie.

Les lectures peuvent être distribuées sur les répliques.

TYPES DE RÉPLICATION

SYNCHRONE

L'écriture est confirmée seulement quand la réplique a reçu les données.

Garantie de cohérence forte. Légèrement plus lente.

ASYNCHRON

L'écriture est confirmée immédiatement. La réplique se met à jour après.
Plus rapide mais risque de perte de données en cas de panne.

FAILOVER AUTOMATIQUE

Si le primaire tombe, une réplique prend automatiquement le relais.
Outils : Patroni (PostgreSQL), MHA (MySQL), Sentinel (Redis).

CONNECTION POOLING

Ouvrir une connexion à une base est coûteux.
Le pool de connexions réutilise les connexions existantes.
Outils : PgBouncer (PostgreSQL), ProxySQL (MySQL), HikariCP (Java).

15.4 MONITORING ET MÉTRIQUES

MÉTRIQUES ESSENTIELLES À SURVEILLER

- Temps de réponse des requêtes (latence P50, P95, P99)
- Nombre de requêtes par seconde (throughput)
- Taux d'utilisation du CPU et de la RAM
- Utilisation des index (ratio hit/miss)
- Nombre de connexions actives vs maximum
- Taille des tables et index (croissance)
- Taux de cache hit (buffer pool)
- Requêtes lentes (slow query log)
- Réplication lag (délai entre primaire et réplique)

OUTILS DE MONITORING

- pg_stat_activity : connexions en cours (PostgreSQL)
- pg_stat_user_tables : stats d'utilisation des tables
- SHOW PROCESSLIST : requêtes en cours (MySQL)
- Prometheus + Grafana : métriques et tableaux de bord
- Datadog, New Relic : monitoring cloud avancé
- pgBadger : analyse des logs PostgreSQL

PARTIE XVI – CAS PRATIQUES ET ARCHITECTURES RÉELLES

16.1 CAS PRATIQUE : SYSTÈME DE GESTION SCOLAIRE (SQL)

SCHÉMA

```
CREATE TABLE etudiants (  
    id SERIAL PRIMARY KEY,  
    matricule VARCHAR(20) UNIQUE NOT NULL,  
    nom VARCHAR(100) NOT NULL,  
    prenom VARCHAR(100) NOT NULL,  
    date_nais DATE,  
    filiere_id INTEGER REFERENCES filieres(id)  
);  
  
CREATE TABLE matieres (  
    id SERIAL PRIMARY KEY,  
    code VARCHAR(10) UNIQUE,  
    nom VARCHAR(100),  
    credits INTEGER DEFAULT 3  
);  
  
CREATE TABLE notes (  
    etudiant_id INTEGER REFERENCES etudiants(id),  
    matiere_id INTEGER REFERENCES matieres(id),  
    semestre VARCHAR(10),  
    note DECIMAL(4,2) CHECK (note BETWEEN 0 AND 20),  
    PRIMARY KEY (etudiant_id, matiere_id, semestre)  
);
```

REQUÊTES UTILES

```
-- Moyenne générale par étudiant
SELECT
    e.nom, e.prenom,
    ROUND(AVG(n.note), 2) AS moyenne_generale,
    CASE WHEN AVG(n.note) >= 10 THEN 'Admis' ELSE 'Ajourné' END AS statut
FROM etudiants e
JOIN notes n ON e.id = n.etudiant_id
WHERE n.semestre = 'S1-2024'
GROUP BY e.id, e.nom, e.prenom
ORDER BY moyenne_generale DESC;

-- Classement par filière avec rang
SELECT
    e.nom, e.prenom, f.nom AS filiere,
    AVG(n.note) AS moyenne,
    RANK() OVER (PARTITION BY e.filiere_id ORDER BY AVG(n.note) DESC) AS rang
FROM etudiants e
JOIN notes n ON e.id = n.etudiant_id
JOIN filieres f ON e.filiere_id = f.id
GROUP BY e.id, e.nom, e.prenom, e.filiere_id, f.nom;
```

16.2 CAS PRATIQUE : RÉSEAU SOCIAL SIMPLIFIÉ (MONGODB)

MODÈLE DE DOCUMENT

```
// Collection "utilisateurs"
{
  "_id": ObjectId("..."),
  "username": "alpha_diallo",
  "nom": "Alpha Diallo",
  "bio": "Data Engineer | Conakry",
  "abonnes": 1240,
  "abonnements": 380,
  "interets": ["data", "technologie", "musique"]
}

// Collection "posts"
{
  "_id": ObjectId("..."),
  "auteur_id": ObjectId("..."),
  "contenu": "Redis sauvegarde vos performances !",
  "date": ISODate("2024-10-15T09:30:00Z"),
  "likes": 87,
  "commentaires": [
    {
      "auteur": "mariama_bah",
      "texte": "Totalement d'accord !",
      "date": ISODate("2024-10-15T10:00:00Z")
    }
  ],
  "hashtags": ["#nosql", "#redis", "#data"]
}
```

REQUÊTES

```
// Posts les plus aimés de la semaine
db.posts.find({
  date: { $gte: new Date("2024-10-09") }
}).sort({ likes: -1 }).limit(10);

// Tous les posts d'un utilisateur
db.posts.find({ auteur_id: ObjectId("...") })
    .sort({ date: -1 });

// Recherche par hashtag
db.posts.find({ hashtags: "#nosql" });
```

16.3 CAS PRATIQUE : DÉTECTION DE FRAUDE (NE04J)

MODÈLE DE GRAPHE

Nœuds : Compte, Personne, Transaction, Appareil, IP
Arêtes : POSSÈDE, EFFECTUE, UTILISE, SE_CONNECTE_DEPUIS

REQUÊTES DE DÉTECTION

```
// Trouver des comptes partageant le même appareil
MATCH (p1:Personne)-[:POSSÈDE]->(c1:Compte),
      (p2:Personne)-[:POSSÈDE]->(c2:Compte),
      (c1)-[:UTILISE]->(d:Appareil)<-[:UTILISE]-(c2)
WHERE p1 <> p2
RETURN p1.nom, p2.nom, d.device_id AS appareil_partage;

// Chaîne de transactions suspectes (argent qui circule vite)
MATCH chemin = (source:Compte)-[:TRANSFERT*3..6]->(dest:Compte)
WHERE ALL(t IN relationships(chemin)
          WHERE t.montant > 10000 AND t.date > datetime() - duration('P1D'))
RETURN source, dest, length(chemin) AS nb_sauts
ORDER BY nb_sauts;
```

16.4 MIGRATION SQL VERS NOSQL : BONNES PRATIQUES

NE PAS MIGRER PAR DÉFAUT

Le NoSQL n'est pas une amélioration du SQL. C'est un outil différent.
Migrer sans raison technique valide est une erreur coûteuse.

SIGNAUX QUI JUSTIFIENT UNE MIGRATION

- La base relationnelle devient un goulot d'étranglement prouvé
- Le schéma change trop souvent pour être géré en SQL
- Un cas d'usage spécialisé apparaît (graphe, cache, séries temporelles)

STRATÉGIE DE MIGRATION PROGRESSIVE

1. Identifier les entités candidates à migrer
2. Implémenter la double écriture (SQL + NoSQL en parallèle)
3. Basculer progressivement la lecture vers le NoSQL
4. Valider la cohérence des données
5. Couper la source SQL pour ce cas d'usage

SYNTHÈSE FINALE – MAÎTRISE DES SGBD

Ce cours couvre l'ensemble du spectre des bases de données modernes.
Voici les axes de maîtrise à développer en priorité :

NIVEAU 1 – FONDAMENTAUX (tout praticien doit maîtriser) ✓ SQL complet (SELECT, JOIN, agrégation, sous-requêtes) ✓ Modélisation relationnelle et normalisation ✓ Transactions et propriétés ACID ✓ Index et optimisation de base
--

NIVEAU 2 – INTERMÉDIAIRE (développeur/data engineer) ✓ Fonctions de fenêtrage et CTE ✓ MongoDB (modélisation document, agrégation) ✓ Redis (structures de données, TTL, pub/sub) ✓ Réplication, backup, monitoring

NIVEAU 3 – AVANCÉ (architecte / DBA) ✓ Théorème CAP et cohérence éventuelle ✓ Cassandra (modélisation orientée requêtes) ✓ Neo4j (modélisation graphe, Cypher) ✓ Architecture polyglot persistance ✓ Scalabilité, sharding, haute disponibilité
--

RÈGLE D'OR DU PRATICIEN :

"Le meilleur SGBD est celui qui répond précisément au besoin du projet.
Connaître les outils ne suffit pas – il faut savoir choisir le bon
au bon moment, pour la bonne raison."

```
=====
GCI – GYKHAMINE CONCEPT INVESTIGATION
COURS SGBD SQL & NoSQL – Version 1.0
© Formation & Intelligence Analytique – Usage pédagogique
=====
```